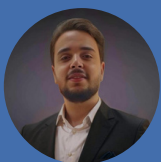
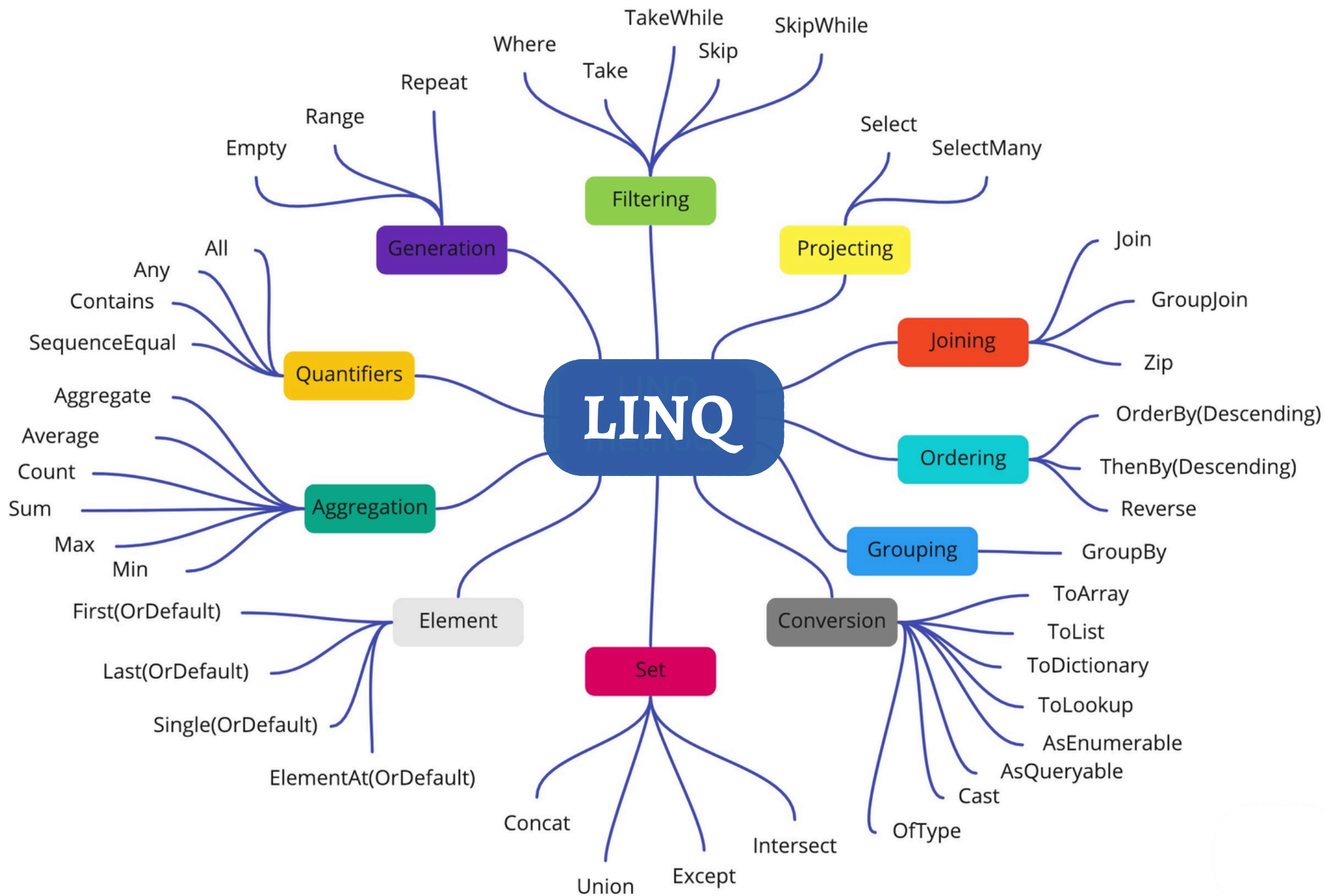


Master LINQ Methods in



Filtering

Where

Take

TakeWhile

Skip

SkipWhile

Where

```
var numbers = new[] { 1, 2, 3, 4, 5 };  
var evenNumbers = numbers.Where(n => n % 2 == 0);  
// Output: { 2, 4 }
```



Take

```
var numbers = new[] { 10, 20, 30, 40, 50 };  
var firstTwo = numbers.Take(2);  
// Output: { 10, 20 }
```

TakeWhile

```
var numbers = new[] { 1, 2, 3, 10, 2 };  
var result = numbers.TakeWhile(n => n < 5);  
// Output: { 1, 2, 3 }
```

Skip

```
var numbers = new[] { 10, 20, 30, 40, 50 };  
var afterTwo = numbers.Skip(2);  
// Output: { 30, 40, 50 }
```

SkipWhile

```
var numbers = new[] { 1, 2, 3, 10, 2 };  
var result = numbers.SkipWhile(n => n < 5);  
// Output: { 10, 2 }
```



Mahmoud Abdelaziz



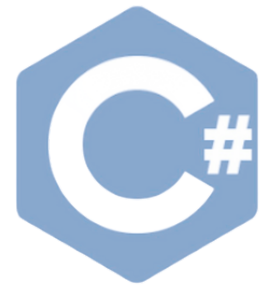
Ordering

● **OrderBy**

● **Reverse**

● **OrderByDesc**

● **ThenBy**



OrderBy

```
var names = new[] { "Charlie", "Alice", "Bob" };
var sorted = names.OrderBy(n => n);
// Output: { "Alice", "Bob", "Charlie" }
```

Reverse

```
var numbers = new[] { 10, 20, 30 };
var reversed = numbers.Reverse();
// Output: { 30, 20, 10 }
```

ThenBy

```
var people = new[]
{
    new { Name = "Ali", Age = 30 },
    new { Name = "Ali", Age = 25 },
    new { Name = "Ziad", Age = 20 }
};

var sorted = people
    .OrderBy(p => p.Name).ThenBy(p => p.Age);
// Output:
// { Name = "Ali", Age = 25 }
// { Name = "Ali", Age = 30 }
// { Name = "Ziad", Age = 20 }
```

OrderByDesc

```
var numbers = new[] { 5, 1, 8, 3 };
var sorted = numbers.OrderByDescending(n => n);
// Output: { 8, 5, 3, 1 }
```





Count



```
var nums = new[] { 1, 2, 3, 4 };  
var total = nums.Count(); // 4  
var evens = nums.Count(n => n % 2 == 0); // 2
```

Sum



```
var nums = new[] { 1, 2, 3 };  
var total = nums.Sum(); // 6
```

Min



```
var nums = new[] { 7, 2, 10 };  
var min = nums.Min(); // 2
```

Max



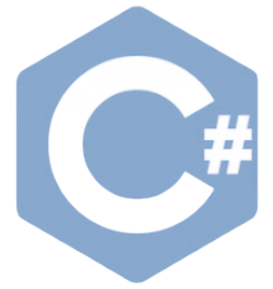
```
var nums = new[] { 7, 2, 10 };  
var max = nums.Max(); // 10
```

Average



```
var nums = new[] { 2, 4, 6 };  
var avg = nums.Average(); // 4
```





Select



```
var numbers = new[] { 1, 2, 3 };  
var squares = numbers.Select(n => n * n);  
// Output: { 1, 4, 9 }
```

SelectMany



```
var nested = new[] {  
    new[] { 1, 2 },  
    new[] { 3 },  
    new[] { 4, 5 }  
};  
  
var flattened = nested.SelectMany(x => x);  
  
// Output: { 1, 2, 3, 4, 5 }
```



All



```
var numbers = new[] { 2, 4, 6 };  
var areAllEven = numbers.All(n => n % 2 == 0);  
// Output: true
```



Any



```
var numbers = new[] { 1, 3, 5 };  
var hasEven = numbers.Any(n => n % 2 == 0);  
// Output: false
```

Contains



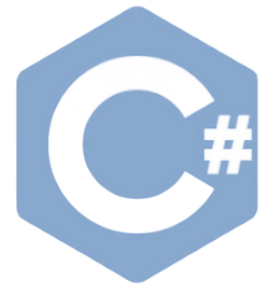
```
var names = new[] { "Alice", "Bob", "Charlie" };  
var hasBob = names.Contains("Bob");  
// Output: true
```

SequenceEqual



```
var list1 = new[] { 1, 2, 3 };  
var list2 = new[] { 1, 2, 3 };  
var areEqual = list1.SequenceEqual(list2);  
// Output: true
```





Concat

```
var list1 = new[] { 1, 2, 3 };
var list2 = new[] { 3, 4, 5 };

var result = list1.Concat(list2);

// Output: { 1, 2, 3, 3, 4, 5 }
```

Union

```
var list1 = new[] { 1, 2, 3 };
var list2 = new[] { 3, 4, 5 };

var result = list1.Union(list2);

// Output: { 1, 2, 3, 4, 5 }
```

Except

```
var list1 = new[] { 1, 2, 3, 4 };
var list2 = new[] { 3, 4, 5 };

var result = list1.Except(list2);

// Output: { 1, 2 }
```

Intersect

```
var list1 = new[] { 1, 2, 3, 4 };
var list2 = new[] { 3, 4, 5 };

var result = list1.Intersect(list2);

// Output: { 3, 4 }
```



● **Generation** ● Repeat ● Range ● Empty

Repeat



```
var repeated = Enumerable.Repeat("Hi", 3);  
  
// Output: { "Hi", "Hi", "Hi" }
```



Range



```
var range = Enumerable.Range(1, 5);  
  
// Output: { 1, 2, 3, 4, 5 }
```

Empty



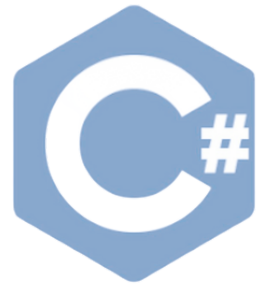
```
var emptyList = Enumerable.Empty<string>();  
  
// Output: {}
```



● **Element** ● **First(OrDefault)** ● **LastOr(Default)**

First

```
var numbers = new[] { 10, 20, 30 };
var first = numbers.First();
// Output: 10
```



FirstOrDefault

```
var numbers = new int[] { };
var firstOrDefault = numbers.FirstOrDefault();
// Output: 0 (default of int)
```

Last

```
var numbers = new[] { 10, 20, 30 };
var last = numbers.Last();
// Output: 30
```

LastOrDefault

```
var numbers = new int[] { };
var lastOrDefault = numbers.LastOrDefault();
// Output: 0
```



● **Element** ● Single(OrDefault) ● ElementAt(OrDefault)

Single

```
var numbers = new[] { 42 };  
var single = numbers.Single();  
// Output: 42
```



SingleOrDefault

```
var numbers = new int[] { };  
var singleOrDefault = numbers.SingleOrDefault();  
// Output: 0
```

ElementAt

```
var numbers = new[] { 100, 200, 300 };  
var element = numbers.ElementAt(1);  
// Output: 200
```

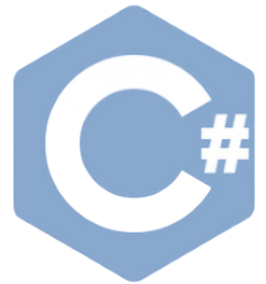
ElementAtOrDefault

```
var numbers = new[] { 100, 200, 300 };  
var element = numbers.ElementAtOrDefault(5);  
// Output: 0 (index is not found)
```



● Grouping ● GroupBy

GroupBy



```
var students = new[] {
    new { Name = "Alice", Class = "Math" },
    new { Name = "Bob", Class = "Science" },
    new { Name = "Charlie", Class = "Math" },
    new { Name = "David", Class = "Science" },
    new { Name = "Eva", Class = "Math" }
};

var grouped = students.GroupBy(s => s.Class);

foreach (var group in grouped)
{
    Console.WriteLine($"Class: {group.Key}");
    foreach (var student in group)
    {
        Console.WriteLine($" - {student.Name}");
    }
}

/* Output
Class: Math
 - Alice
 - Charlie
 - Eva
Class: Science
 - Bob
 - David */
```



● Conversion ● ToArray ● ToList ● ToDictionary

ToArray

```
var numbers = Enumerable.Range(1, 3); // {1, 2, 3}
var array = numbers.ToArray();
// Output: array = int[]
```



ToList

```
var numbers = Enumerable.Range(1, 3); // {1, 2, 3}
var list = numbers.ToList();
// Output: list = List<int>
```

ToDictionary

```
var people = new[]{
    new { Id = 1, Name = "Alice" },
    new { Id = 2, Name = "Bob" }
};

var dictionary = people.
    ToDictionary(p => p.Id, p => p.Name);

// Output: { [1] = "Alice", [2] = "Bob" }
```



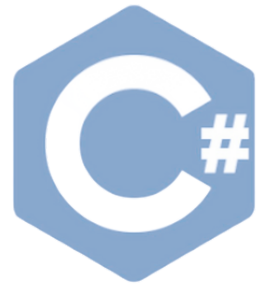
● Conversion

● ToLookup

● AsEnumerable

● AsQueryable

ToLookup



```
var items = new[]{
    new { Category = "Fruit", Name = "Apple" },
    new { Category = "Fruit", Name = "Banana" },
    new { Category = "Vegetable", Name = "Carrot" }};

var lookup = items.ToLookup(i => i.Category);

// Output:
// Fruit => Apple, Banana
// Vegetable => Carrot
```

AsEnumerable

```
DataTable table = GetDataTable();
var rows = table.AsEnumerable();

// Output: IEnumerable<DataRow>
```

AsQueryable

```
List<int> list = new List<int> { 1, 2, 3 };
var queryable = list.AsQueryable();

// Output: IQueryable<int>
```

